反向传播全流程：从 Loss 到每一个参数梯度

> **原始问题：**从头到尾详细讲解一次反向传播的过程，并且要简单的例子来说明，包含具体数值和shape，使用图片来辅助我完整理解整个流程

提问：2026/08/13 11:14　更新：2026/08/13 11:24　重要度：5/5

反向传播全流程：从 Loss 到每一个参数梯度

一句话结论

反向传播（Backpropagation）不是一种独立于链式法则的神秘算法，而是：在前向计算形成的计算图上，从标量 Loss 开始，按照与前向相反的拓扑顺序，把上游梯度乘以每个算子的局部导数，并在分叉汇合处累加，最终得到每个参数的梯度。

完整训练 Step 可以压缩为：

```
前向：输入 X → 各层中间激活 → 预测 Y_hat → Loss L
建图：记录哪些 Tensor 由哪些算子产生，以及 backward 需要的中间值
反向：从 dL/dL = 1 出发，逐算子执行 Vector-Jacobian Product
累加：同一 Tensor 经多条路径影响 Loss 时，把各路径贡献相加
更新：Optimizer 使用 parameter.grad 修改参数
清理：梯度清零或置 None，释放不再需要的计算图和激活
```

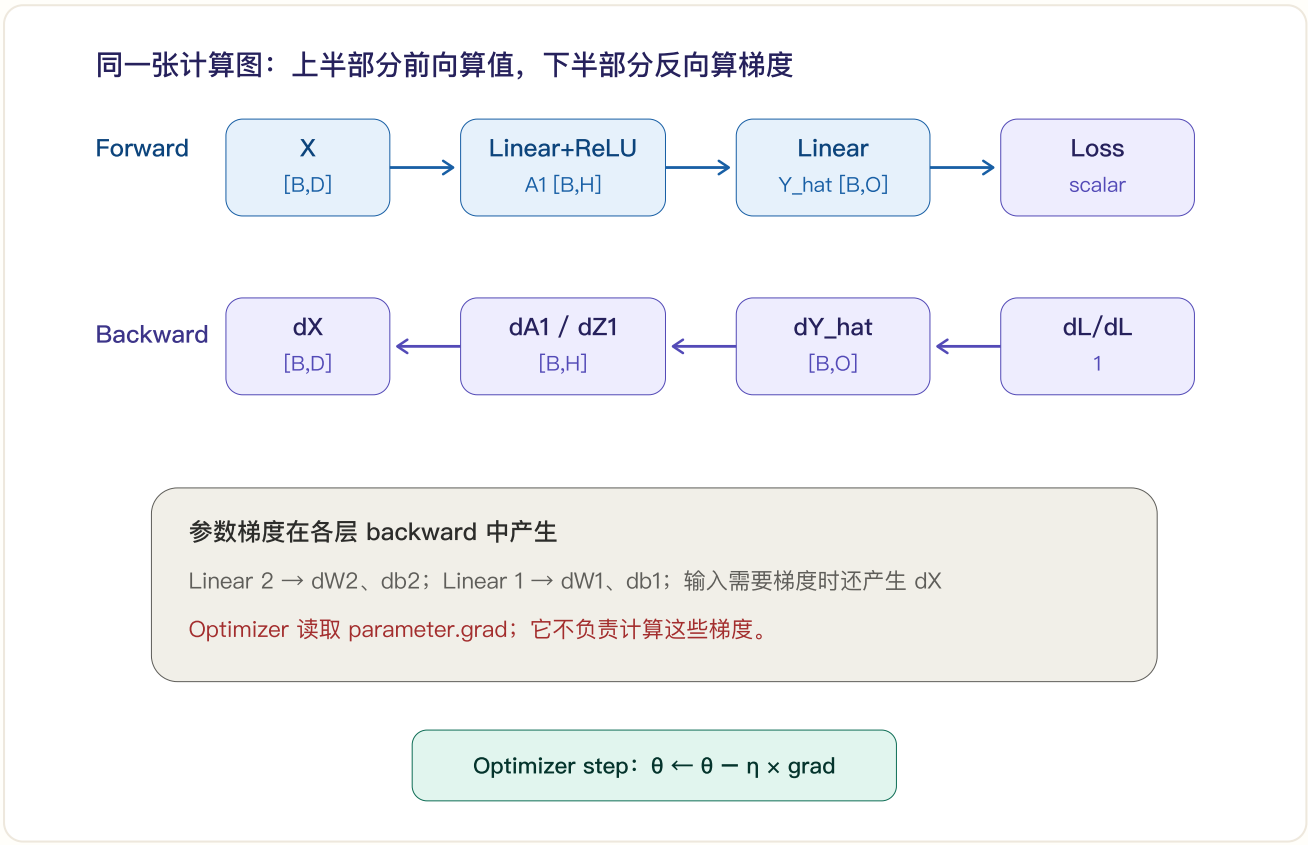
最重要的三个心智模型：

1. **梯度表示敏感度**：`∂L/∂x` 表示 `x` 发生一个微小变化时，Loss 会怎样变化；
2. **每个算子只做局部工作**：Backward 不需要一次构造整个网络的巨大 Jacobian；每个节点只接收上游梯度并返回对输入的梯度；
3. **反向传播的是梯度，不是原始激活**：但计算梯度通常需要读取前向保存的输入、输出、Mask 或统计量。

一、反向传播在训练流程中的位置

一次标准训练迭代包含：

```
1. 读取一个 Mini-batch
2. Forward：根据当前参数产生预测
3. Loss：比较预测与目标
4. Backward：计算 Loss 对所有可训练参数的梯度
5. Gradient Processing：Unscale、Clip、分布式同步
6. Optimizer Step：使用梯度更新参数
7. Zero Grad：清理梯度，进入下一次迭代
```



二、先用一个标量例子理解链式法则

设一个最小模型：

```
x = 2
w = 3
b = 1
y = 8
```

前向：

```
z = wx + b
  = 3×2 + 1
  = 7

L = 1/2 × (z−y)²
  = 1/2 × (7−8)²
  = 0.5
```

我们需要：

```
∂L/∂w
∂L/∂b
∂L/∂x
```

把计算拆成局部关系：

```
L = 1/2 × e²
e = z−y
z = wx+b
```

局部导数：

```
∂L/∂e = e = −1
∂e/∂z = 1
∂z/∂w = x = 2
∂z/∂b = 1
∂z/∂x = w = 3
```

从 Loss 反向：

```
∂L/∂L = 1

∂L/∂z
= ∂L/∂e × ∂e/∂z
= −1 × 1
= −1
```

再传播到 `w,b,x`：

```
∂L/∂w
= ∂L/∂z × ∂z/∂w
= −1 × 2
= −2

∂L/∂b
= ∂L/∂z × ∂z/∂b
= −1 × 1
= −1

∂L/∂x
= ∂L/∂z × ∂z/∂x
= −1 × 3
= −3
```

梯度符号的含义：

```
∂L/∂w = -2
```

表示 `w` 微增时，Loss 会下降。使用 SGD：

```
w_new = w - η × ∂L/∂w
```

若 `η=0.1`：

```
w_new = 3 - 0.1×(-2) = 3.2
```

负梯度导致参数增大，这与当前预测 `7` 小于目标 `8` 一致。

三、计算图、局部梯度和上游梯度

每个计算图节点只需要知道三件事：

```
1. 自己的输入和输出
2. 自己的局部导数
3. 从下游收到的上游梯度
```

假设：

```
a = f(x)
L = g(a)
```

反向公式：

```
∂L/∂x
= ∂L/∂a × ∂a/∂x
```

可以读成：

```
对 x 的总敏感度
= 下游 Loss 对 a 的敏感度
× 当前算子 a 对 x 的局部敏感度
```

为什么反向从标量 Loss 的 1 开始

Loss 是标量：

```
L ∈ R
```

它对自身的导数：

```
∂L/∂L = 1
```

这个 `1` 是反向传播的初始种子。之后每个节点把上游梯度转换成对自己输入的梯度。

为什么变量的梯度通常和变量 Shape 相同

若：

```
X: [B,D]
```

则：

```
dX = ∂L/∂X: [B,D]
```

`dX[i,j]` 表示 `X[i,j]` 微小变化时，标量 Loss 的变化率。

参数也一样：

```
W:  [D,H]
dW: [D,H]
```

这条规则是检查 Backward Shape 是否正确的第一道 sanity check。

四、统一数值例子：两层 MLP

全文使用同一组参数：

```
Batch Size      B = 2
Input Dimension D = 2
Hidden Size     H = 2
Output Size     O = 1
```

网络：

```
Z1    = XW1 + b1
A1    = ReLU(Z1)
```

```
Y_hat = A1W2 + b2
L     = (1/(2B)) × Σ(Y_hat−Y)²
```

Shape：

```
X      [B,D] = [2,2]
W1     [D,H] = [2,2]
b1     [H]   = [2]
Z1/A1  [B,H] = [2,2]
W2     [H,O] = [2,1]
b2     [O]   = [1]
Y_hat  [B,O] = [2,1]
Y      [B,O] = [2,1]
L      scalar
```

具体数值：

```
X = [[ 1, 2],
     [−1, 1]]

W1 = [[ 0.5, −1.0],
      [ 1.0,  0.5]]

b1 = [0.0, −0.5]

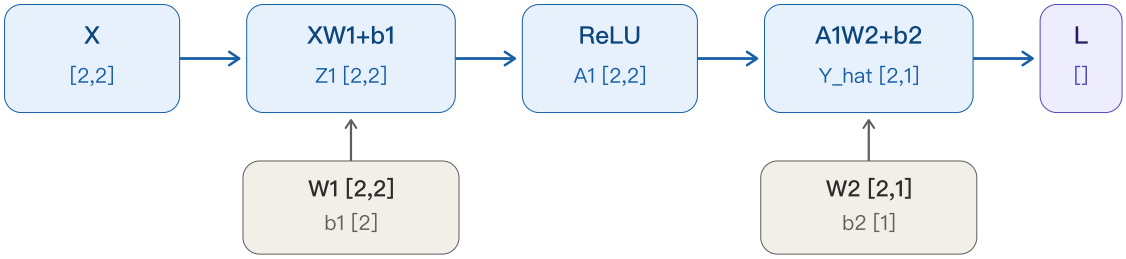
W2 = [[ 0.4],
      [−0.6]]

b2 = [0.1]

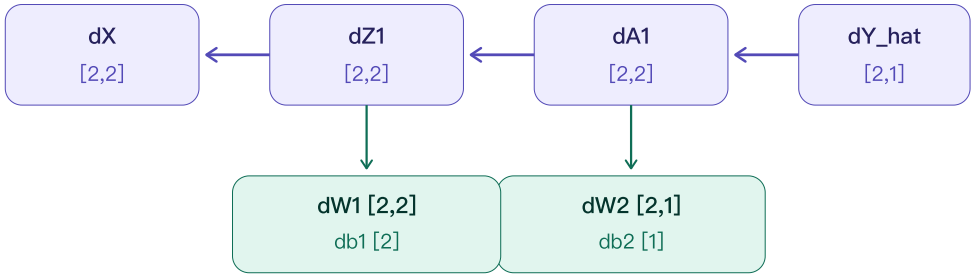
Y = [[1.0],
     [0.0]]
```

统一例子：B=2，D=2，H=2，O=1

Forward values



Backward gradients



每个梯度与被求导变量 Shape 相同；矩阵转置来自维度对齐。

五、前向传播：从 X 算到 Loss

5.1 第一层线性变换

```
Z1 = XW1 + b1
```

Shape 检查：

```
X   [2,2]
W1  [2,2]
XW1 [2,2]
b1  [2] → Broadcast 成每一行都加 [0,-0.5]
Z1  [2,2]
```

计算 `XW1`：

```
XW1
= [[ 1, 2],     [[ 0.5, -1.0],
   [-1, 1]]  ×   [ 1.0,  0.5]]
```

```
= [[1×0.5+2×1.0, 1×(−1.0)+2×0.5],
   [(−1)×0.5+1×1.0, (−1)×(−1.0)+1×0.5]]

= [[2.5, 0.0],
   [0.5, 1.5]]
```

加偏置：

```
Z1
= [[2.5, 0.0],
   [0.5, 1.5]]
+ [0.0, −0.5]

= [[2.5, −0.5],
   [0.5,  1.0]]
```

5.2 ReLU

```
A1 = ReLU(Z1) = max(0,Z1)
```

逐元素：

```
A1
= [[2.5, 0.0],
   [0.5, 1.0]]
```

注意第一行第二个值：

```
Z1[0,1] = −0.5
A1[0,1] = 0
```

它会在反向时把该位置的梯度截断为 0。

5.3 第二层线性变换

```
Y_hat = A1W2 + b2
```

Shape：

```
A1     [2,2]
W2     [2,1]
A1W2   [2,1]
b2     [1]
Y_hat  [2,1]
```

数值：

```
A1W2
= [[2.5,0.0],     [[ 0.4],
   [0.5,1.0]]  ×   [-0.6]]

= [[2.5×0.4 + 0×(-0.6)],
   [0.5×0.4 + 1×(-0.6)]]

= [[ 1.0],
   [-0.4]]
```

加 `b2=0.1`：

```
Y_hat
= [[ 1.1],
   [-0.3]]
```

5.4 MSE Loss

定义：

```
E = Y_hat - Y

L = (1/(2B)) × Σ E²
```

这里使用 `1/(2B)`，前面的 `1/2` 会与平方求导产生的 `2` 抵消。

误差：

```
E
= [[ 1.1],      [[1.0],
   [-0.3]]  -    [0.0]]

= [[ 0.1],
   [-0.3]]
```

Loss：

```
L
= 1/(2×2) × (0.1² + (-0.3)²)
= 1/4 × (0.01 + 0.09)
= 0.025
```

前向完成后，计算图至少需要为反向保留这些信息：

```
X、W1：计算 dW1 和 dX
Z1 或 ReLU Mask：计算 dZ1
A1、W2：计算 dW2 和 dA1
Y_hat、Y 或 E：计算 dY_hat
```

六、反向第 1 步：Loss 对预测的梯度

```
L = (1/(2B)) × Σ(Y_hat−Y)²
```

对 `Y_hat` 求导：

```
dY_hat
= ∂L/∂Y_hat
= (Y_hat−Y) ÷ B
= E ÷ B
```

因为 `B=2`：

```
dY_hat
= [[ 0.1],
   [−0.3]] ÷ 2

= [[ 0.05],
   [−0.15]]
```

Shape：

```
Y_hat   [2,1]
dY_hat  [2,1]
```

含义：

- 第一个样本预测偏高 `0.1`，梯度为正；梯度下降会尝试降低预测；
- 第二个样本预测偏低 `−0.3`，梯度为负；梯度下降会尝试提高预测。

七、反向第 2 步：第二个 Linear Layer

前向：

```
Y_hat = A1W2 + b2
```

收到上游梯度：

```
dY_hat: [B,O] = [2,1]
```

Linear Backward 的三个输出：

```
dW2 = A1^T × dY_hat
db2 = sum_batch(dY_hat)
dA1 = dY_hat × W2^T
```

7.1 为什么 dW2 要用 A1 转置

Shape：

```
A1^T       [H,B] = [2,2]
dY_hat     [B,O] = [2,1]
dW2        [H,O] = [2,1]
```

矩阵乘法收缩 Batch 轴：

```
dW2[h,o]
= Σ_b A1[b,h] × dY_hat[b,o]
```

每个样本对同一共享参数 `W2` 的梯度贡献要在 Batch 上求和。

数值：

```
dW2
= [[2.5,0.5],     [[ 0.05],
   [0.0,1.0]]  ×   [-0.15]]

= [[2.5×0.05 + 0.5×(-0.15)],
   [0.0×0.05 + 1.0×(-0.15)]]

= [[ 0.05],
   [-0.15]]
```

7.2 偏置梯度

`b2` 被 Broadcast 到每个样本，所以反向必须沿被 Broadcast 的 Batch 轴求和：

```
db2
= 0.05 + (-0.15)
= -0.10
```

Shape：

```
b2   [1]
db2  [1]
```

7.3 传回隐藏层的梯度

```
dA1 = dY_hat × W2^T
```

Shape：

```
dY_hat  [B,O] = [2,1]
W2^T     [O,H] = [1,2]
dA1      [B,H] = [2,2]
```

数值：

```
dA1
= [[ 0.05],     [[0.4, −0.6]]
   [−0.15]]  ×

= [[ 0.05×0.4,  0.05×(−0.6)],
   [−0.15×0.4, −0.15×(−0.6)]]

= [[ 0.02, −0.03],
   [−0.06,  0.09]]
```

八、反向第 3 步：ReLU

前向：

```
A1 = ReLU(Z1)
```

局部导数：

```
ReLU'(z) = 1, z>0
ReLU'(z) = 0, z<0
```

在 `z=0` 处数学上不可导，框架通常选择一个约定值，常见实现取 0。

本例：

```
Z1
= [[2.5, −0.5],
   [0.5,  1.0]]
```

Mask：

```
M = (Z1>0)
```

```
= [[1,0],
   [1,1]]
```

反向逐元素相乘：

```
dZ1 = dA1 ⊙ M
```

```
dZ1
= [[ 0.02, -0.03],     [[1,0],
   [-0.06,  0.09]]  ⊙   [1,1]]

= [[ 0.02, 0.00],
   [-0.06, 0.09]]
```

关键点：

```
dA1[0,1] = -0.03
```

但前向 `Z1[0,1] = -0.5` 被 ReLU 截断，因此：

```
dZ1[0,1] = 0
```

这就是 ReLU 的梯度门控。

九、反向第 4 步：第一个 Linear Layer

前向：

```
Z1 = XW1 + b1
```

Backward：

```
dW1 = X^T × dZ1
db1 = sum_batch(dZ1)
dX  = dZ1 × W1^T
```

9.1 计算 dW1

Shape：

```
X^T   [D,B] = [2,2]
dZ1    [B,H] = [2,2]
```

```
dW1    [D,H] = [2,2]
```

```
X^T
= [[1,-1],
   [2, 1]]
```

计算：

```
dW1
= [[1,-1],     [[ 0.02,0.00],
   [2, 1]]  ×   [-0.06,0.09]]
```

逐项：

```
dW1[0,0] = 1×0.02 + (-1)×(-0.06) = 0.08
dW1[0,1] = 1×0    + (-1)×0.09    = -0.09
dW1[1,0] = 2×0.02 + 1×(-0.06)    = -0.02
dW1[1,1] = 2×0    + 1×0.09       = 0.09
```

所以：

```
dW1
= [[ 0.08, -0.09],
   [-0.02,  0.09]]
```

9.2 计算 db1

沿 Batch 轴求和：

```
db1
= [0.02 + (-0.06), 0 + 0.09]
= [-0.04, 0.09]
```

9.3 计算 dX

如果 `X` 来自更前面的网络层，就必须继续传播 `dX`；若 `X` 是原始数据且不需要输入梯度，可以不保存或不返回它。

```
dX = dZ1 × W1^T
```

```
W1^T
= [[ 0.5, 1.0],
   [-1.0, 0.5]]
```

```
dX
= [[ 0.02,0.00],     [[ 0.5,1.0],
   [-0.06,0.09]]  ×   [-1.0,0.5]]
```

```
= [[ 0.01,  0.02],
   [-0.12, -0.015]]
```

Shape：

```
X   [2,2]
dX  [2,2]
```

十、全部梯度汇总

变量	Shape	数值
`dY_hat`	`[2,1]`	`[[0.05],[-0.15]]`
`dW2`	`[2,1]`	`[[0.05],[-0.15]]`
`db2`	`[1]`	`[-0.10]`
`dA1`	`[2,2]`	`[[0.02,-0.03],[-0.06,0.09]]`
`dZ1`	`[2,2]`	`[[0.02,0],[-0.06,0.09]]`
`dW1`	`[2,2]`	`[[0.08,-0.09],[-0.02,0.09]]`
`db1`	`[2]`	`[-0.04,0.09]`
`dX`	`[2,2]`	`[[0.01,0.02],[-0.12,-0.015]]`

Sanity Check：

```
梯度 Shape = 被求导变量 Shape

dW1.shape == W1.shape == [2,2]
db1.shape == b1.shape == [2]
dW2.shape == W2.shape == [2,1]
db2.shape == b2.shape == [1]
dX.shape  == X.shape  == [2,2]
```

十一、Optimizer 如何使用这些梯度

假设使用 SGD：

```
η = 0.1
```

```
θ_new = θ − η × dθ
```

更新 `W2`：

```
W2_old
= [[ 0.4],
   [-0.6]]

dW2
= [[ 0.05],
   [-0.15]]

W2_new
= W2_old − 0.1×dW2

= [[0.4-0.005],
   [-0.6-(-0.015)]]

= [[ 0.395],
   [-0.585]]
```

更新 `b2`：

```
b2_new
= 0.1 − 0.1×(−0.1)
= 0.11
```

更新 `W1`：

```
W1_new
= [[ 0.5,-1.0],
   [ 1.0, 0.5]]
− 0.1 ×
  [[ 0.08,-0.09],
   [-0.02, 0.09]]

= [[0.492,-0.991],
   [1.002, 0.491]]
```

更新 `b1`：

```
b1_new
= [0.0,-0.5] − 0.1×[-0.04,0.09]
= [0.004,-0.509]
```

若重新 Forward，新 Loss 应略微下降。反向传播只计算梯度；真正修改参数的是 Optimizer。

十二、为什么梯度在分叉处要累加

考虑：

```
y = x² + 3x
```

`x` 通过两条路径影响 `y`：

```
路径 1: x → x² → y
路径 2: x → 3x → y
```

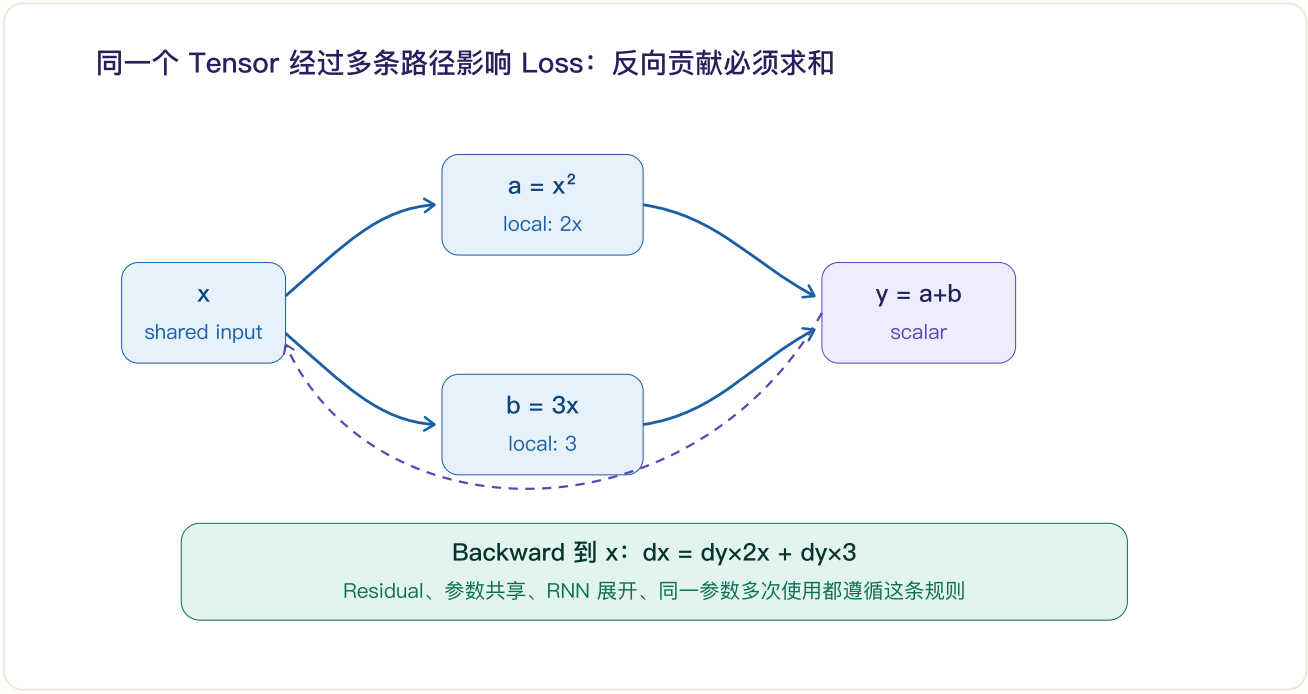
两条路径贡献：

```
路径 1: ∂y/∂x = 2x
路径 2: ∂y/∂x = 3
```

总梯度：

```
∂y/∂x = 2x + 3
```

不是二选一，而是相加。



神经网络中的典型例子：

- Residual：`y=x+F(x)`，`x` 同时经过 Identity 和 `F`；
- Weight Sharing：同一个参数在多个位置被使用；
- RNN：同一参数跨时间步共享；
- 一个 Tensor 同时供多个后续算子使用。

Autograd 会等所有下游贡献到达后，对同一叶子 Tensor 的 `.grad` 做累加。

十三、广播、Reduce 和反向 Shape

前向中的 Broadcast，反向通常对应 Reduce-Sum。

本例：

```
Z1 = XW1 + b1
```

`b1:[H]` 在前向被 Broadcast 到 `[B,H]`：

```
每个样本都加同一个 b1
```

反向：

```
db1[h] = Σ_b dZ1[b,h]
```

所以 Batch 轴消失。

反过来，前向 Reduce-Sum 的反向通常是 Broadcast：

```
s = Σ_i x_i
```

若上游梯度为 `ds`：

```
dx_i = ds
```

每个输入元素都收到同一个上游梯度。

理解 Broadcast/Reduce 的互逆关系，可以解释很多 Shape Bug。

十四、反向传播为何不显式构造完整 Jacobian

如果：

```
f: R^n → R^m
```

完整 Jacobian：

```
J_f: [m,n]
```

大型网络的 `m,n` 都可能极大，显式构造不可接受。

反向传播真正计算的是 Vector-Jacobian Product（VJP）：

```
v^T × J_f
```

其中 `v` 是从下游传来的上游梯度。

对标量 Loss：

```
L: R^m → R
```

我们只关心：

```
∂L/∂x
= (∂L/∂f) × (∂f/∂x)
```

而不需要单独得到 `∂f/∂x` 的全部矩阵元素。

这也是 Reverse-Mode Automatic Differentiation 特别适合神经网络训练的原因：

```
输入/参数很多
输出 Loss 通常只有一个标量
```

一次反向就能得到 Loss 对所有参数的梯度。

十五、Autograd 实际保存了什么

以 PyTorch 动态计算图为例：

```
前向执行算子
→ 产生 Tensor
→ 为需要梯度的结果记录 grad_fn
→ grad_fn 保存 backward 所需的上下文
```

不同算子需要保存不同内容：

算子	Backward 常需保存
Linear/MatMul	输入、权重或其必要视图

算子	Backward 常需保存
ReLU	输入符号或输出 Mask
Sigmoid	输出值，可由 `y(1−y)` 求导
Softmax	Softmax 输出
LayerNorm	输入、均值、逆标准差、Scale
Dropout	随机 Mask 或可复现 RNG 状态
Attention	Q/K/V、概率或重计算所需统计量，取决于实现

因此训练显存不仅有参数和梯度，还有 Saved Activations。

为什么 `torch.no_grad()` 会省显存

因为在 `no_grad` 下：

```
不建立反向图
不保存 backward 上下文
```

所以推理显存通常显著低于训练。

为什么执行完 Backward 后图通常被释放

默认 `loss.backward()` 后，Autograd 会释放许多中间 Saved Tensors。若要对同一图再次 Backward，需要：

```
retain_graph=True
```

但这会延长激活生命周期，显著增加显存，不应无目的使用。

十六、Gradient Accumulation 的两种含义

“梯度累加”容易混淆成两件事。

16.1 计算图分叉贡献累加

同一个 Tensor 通过多条路径影响 Loss：

```
dx = dx_path1 + dx_path2 + ...
```

这是链式法则本身要求的。

16.2 多个 Micro-batch 的 `.grad` 累加

PyTorch 默认不会在 Backward 前自动清空叶子参数的 `.grad`：

```
loss1.backward()
loss2.backward()
```

执行后：

```
param.grad
= grad_from_loss1 + grad_from_loss2
```

这使得显存不足时可以把一个大 Batch 拆成多个 Micro-batches。

若希望等价于大 Batch 的平均 Loss，通常每个 Micro-batch Loss 要除以累加步数：

```
loss = loss / accumulation_steps
loss.backward()
```

否则梯度总量会扩大 `accumulation_steps` 倍，相当于改变有效学习率。

为什么每个 Optimizer Step 后要 Zero Grad

```
optimizer.zero_grad(set_to_none=True)
```

如果忘记，下一训练 Step 会继续累加旧梯度。

`set_to_none=True` 与写零的区别：

- `None` 可以避免一遍清零内存写；
- 下一次 Backward 直接分配/写入梯度；
- 某些检查代码需要处理 `.grad is None`。

十七、用 PyTorch 验证全部手算

```
import torch

torch.set_printoptions(precision=6, sci_mode=False)
```

```
X = torch.tensor(
    [[ 1.0, 2.0],
     [-1.0, 1.0]],
    requires_grad=True,
)

W1 = torch.tensor(
    [[ 0.5, -1.0],
     [ 1.0,  0.5]],
    requires_grad=True,
)

b1 = torch.tensor(
    [0.0, -0.5],
    requires_grad=True,
)

W2 = torch.tensor(
    [[ 0.4],
     [-0.6]],
    requires_grad=True,
)

b2 = torch.tensor(
    [0.1],
    requires_grad=True,
)

Y = torch.tensor(
    [[1.0],
     [0.0]],
)

# Forward
Z1 = X @ W1 + b1
A1 = torch.relu(Z1)
Y_hat = A1 @ W2 + b2
B = X.shape[0]
loss = ((Y_hat - Y) ** 2).sum() / (2 * B)

# 保留非叶子 Tensor 的梯度，仅用于教学观察
Z1.retain_grad()
A1.retain_grad()
Y_hat.retain_grad()

# Backward
loss.backward()

print("Z1 =\n", Z1)
print("A1 =\n", A1)
print("Y_hat =\n", Y_hat)
print("loss =", loss.item())

print("dY_hat =\n", Y_hat.grad)
print("dW2 =\n", W2.grad)
print("db2 =\n", b2.grad)
print("dA1 =\n", A1.grad)
print("dZ1 =\n", Z1.grad)
print("dW1 =\n", W1.grad)
print("db1 =\n", b1.grad)
print("dX =\n", X.grad)
```

期望结果：

```
loss = 0.025

dY_hat = [[ 0.05],
          [-0.15]]

dW2 = [[ 0.05],
```

```
       [-0.15]]

db2 = [-0.10]

dA1 = [[ 0.02, -0.03],
       [-0.06,  0.09]]

dZ1 = [[ 0.02, 0.00],
       [-0.06, 0.09]]

dW1 = [[ 0.08, -0.09],
       [-0.02,  0.09]]

db1 = [-0.04, 0.09]

dX = [[ 0.01,  0.02],
      [-0.12, -0.015]]
```

为什么普通非叶子 Tensor 默认没有 `.grad`

在 PyTorch 中：

- 叶子参数的 `.grad` 默认保留，Optimizer 需要读取；
- 中间 Tensor 的梯度通常只在反向过程中临时流过，不默认保存在 `.grad`；
- 教学或调试时可调用 `retain_grad()`。

这能避免为所有中间 Tensor 长期保存梯度。

十八、用有限差分检查梯度

解析梯度可以通过数值梯度验证。

对参数 `w`：

```
∂L/∂w
≈ [L(w+ε) - L(w-ε)] ÷ (2ε)
```

这是中心差分。

例如检查 `W1[0,0]`：

```
with torch.no_grad():
    eps = 1e-4
    original = W1[0, 0].item()

    W1[0, 0] = original + eps
    loss_plus = ((((torch.relu(X @ W1 + b1) @ W2 + b2) - Y) ** 2).sum() / (2 * B))

    W1[0, 0] = original - eps
    loss_minus = ((((torch.relu(X @ W1 + b1) @ W2 + b2) - Y) ** 2).sum() / (2 * B))
```

```
    W1[0, 0] = original

numerical = (loss_plus - loss_minus) / (2 * eps)
print("autograd:", W1.grad[0, 0].item())  # 约 0.08
print("numerical:", numerical.item())     # 应接近 0.08
```

有限差分的边界：

- ε 太大：截断误差大；
- ε 太小：浮点舍入误差大；
- ReLU 在 0 附近不可导，扰动可能跨过分段边界；
- 大模型不可能逐参数检查，只适合小模块和随机抽样；
- FP16/BF16 不适合严格 Grad Check，通常使用 FP64 小规模验证。

十九、Mixed Precision 下反向传播多了什么

BF16/FP16 训练不改变链式法则，但会改变数值表示和状态精度。

FP16 Loss Scaling

FP16 的小梯度可能下溢到 0，因此先放大 Loss：

```
L_scaled = S × L
```

Backward 得到：

```
g_scaled = S × g
```

Optimizer Step 前再 Unscale：

```
g = g_scaled ÷ S
```

然后检查 Inf/NaN、执行 Gradient Clipping 和参数更新。

顺序必须正确：

```
Backward scaled loss
→ Unscale gradients
→ Inf/NaN check
→ Gradient clipping
→ Optimizer step
```

若在 Unscale 前 Clip，裁剪的是放大后的梯度，阈值语义会错误。

BF16

BF16 指数范围接近 FP32，通常不需要动态 Loss Scaling，但尾数精度较低。具体算子仍可能使用 FP32 Accumulation。

二十、分布式训练中的反向传播

DDP

每个 Rank 对自己的 Local Batch 做 Forward/Backward，得到本地参数梯度：

```
g_r = ∂L_r/∂θ
```

DDP 对参数梯度执行 AllReduce/Average：

```
g_global
= (1/DP) × Σ_r g_r
```

然后每个 Rank 使用相同的全局梯度更新相同参数。

为了通信计算重叠，Autograd 在某个 Gradient Bucket 准备完成后就可启动异步 AllReduce，同时继续计算更前面层的 Backward。

FSDP / ZeRO

参数、梯度或优化器状态被分片：

- Forward/Backward 前按需 AllGather 参数；
- Backward 计算局部梯度；
- 用 ReduceScatter 聚合并分片梯度；
- 每 Rank 只保留或更新自己负责的状态分片。

链式法则没有变化，改变的是 Tensor 布局、通信时机和生命周期。

Pipeline Parallel

每个 Stage 只保存自己层的激活和参数。Backward 时：

```
后一个 Stage 计算本 Stage 的输入梯度
→ P2P Send 到前一个 Stage
→ 前一个 Stage 把它作为上游梯度继续 Backward
```

跨 Stage 发送的 Activation Gradient，本质上就是链式法则中的上游梯度。

二十一、Activation Checkpointing 为什么能省显存

普通训练：

```
Forward 保存中间激活
Backward 直接读取激活计算梯度
```

Activation Checkpointing：

```
Forward 只保存少量边界激活
Backward 到该区间时重新执行一次 Forward
再利用重算出的中间值做 Backward
```

它做的交换：

```
更少激活显存
↔ 更多重计算 FLOP 和时间
```

它不改变理论梯度，但要注意：

- Dropout/RNG 状态必须正确恢复；
- 有状态算子和副作用需要特殊处理；
- 重计算区间选择影响显存收益和额外计算；
- Mixed Precision 与算子非确定性可能导致微小数值差异。

二十二、反向传播的性能瓶颈

Backward 往往比 Forward 更复杂，因为一个算子通常需要计算：

```
输入梯度 dX
权重梯度 dW
偏置梯度 db
```

以 Linear 为例：

```
Forward:  Y = XW
Backward: dX = dY W^T
          dW = X^T dY
```

Forward 一个主 GEMM，Backward 通常至少两个主 GEMM，所以训练计算成本显著高于只做 Forward 的推理。

主要瓶颈包括：

- GEMM/Attention Backward 计算；
- Saved Activation 读写 HBM；
- Gradient Tensor 写回；
- DDP AllReduce 或 FSDP ReduceScatter；
- Activation Checkpoint 重算；
- 小算子和 Kernel Launch；
- Gradient Accumulation 的额外迭代；
- Optimizer State 的 HBM 访问。

优化时需要区分：

```
Backward Compute Time
Gradient Communication Time
Exposed Communication Time
Activation Recompute Time
Optimizer Step Time
```

二十三、常见失败模式与误区

误区 1：反向传播是在“把 Loss 传回去”

更准确地说，传回去的是：

```
Loss 对各个中间变量的梯度
```

不是 Loss 数值本身。

误区 2：每层直接算 Loss 对参数的导数

实现上每层只需要：

```
上游梯度 × 局部导数
```

不需要知道整个下游网络细节。

误区 3：矩阵转置只是记忆公式

转置来自 Shape 和收缩轴：

```
Y = XW

dW = X^T dY
dX = dY W^T
```

先写 Shape，再推公式，比背公式可靠。

误区 4：一个 Tensor 只能收到一份梯度

如果有多条下游路径，梯度必须相加。Residual 和参数共享都依赖这一点。

误区 5：`loss.backward()` 会自动清空旧梯度

不会。叶子参数 `.grad` 默认累加，必须由训练循环显式清理。

误区 6：梯度正确就一定训练稳定

即使 Backward 数学正确，训练仍可能因以下因素失败：

- Learning Rate 过大；
- Loss Scaling 错误；
- Gradient Explosion；
- 数据或 Label 错误；
- 分布式 Collective 不一致；
- 低精度溢出；
- Optimizer 状态损坏；
- 错误的 Mask、Reduction 或 Loss 归一化。

误区 7：Backward 必须保存所有前向结果

不一定。可以只保存必要统计量，或使用 Activation Checkpointing 重算。

二十四、从头到尾再串一次

使用本例，完整数据流为：

```
X [2,2]
  ↓ Linear 1: XW1+b1
Z1 [[2.5,-0.5],[0.5,1.0]]
  ↓ ReLU
A1 [[2.5,0],[0.5,1.0]]
  ↓ Linear 2: A1W2+b2
Y_hat [[1.1],[-0.3]]
  ↓ MSE with Y
L = 0.025
```

从标量种子开始：

```
dL/dL = 1
```

反向：

```
MSE backward
→ dY_hat = [[0.05],[-0.15]]

Linear 2 backward
→ dW2 = [[0.05],[-0.15]]
→ db2 = [-0.10]
→ dA1 = [[0.02,-0.03],[-0.06,0.09]]

ReLU backward
→ Mask = [[1,0],[1,1]]
→ dZ1 = [[0.02,0],[-0.06,0.09]]

Linear 1 backward
→ dW1 = [[0.08,-0.09],[-0.02,0.09]]
→ db1 = [-0.04,0.09]
→ dX = [[0.01,0.02],[-0.12,-0.015]]
```

Optimizer：

```
读取 W1.grad、b1.grad、W2.grad、b2.grad
→ 更新参数
→ 清理 grad
→ 下一 Batch 使用新参数重新 Forward
```

这就是一次完整反向传播。

可选自测

1. Shape 题

若：

```
X:[B,D]
W:[D,H]
Y=XW:[B,H]
dY:[B,H]
```

请只根据 Shape 推出 `dW` 和 `dX` 的矩阵乘法表达式。

2. 梯度累加题

对于：

```
y = x² + 3x
L = y²
```

当 `x=2` 时，手算 `dL/dx`。注意 `x` 有两条路径到达 `y`。

3. 工程题

为什么 DDP 可以在最后几层的梯度计算完成后立即启动 AllReduce，同时 GPU 继续计算前面层的 Backward？它依赖什么数据依赖条件？

继续学习

1. 前置：矩阵微积分与 VJP/JVP —— 从 Shape 出发严格推导 Linear、Softmax 和 LayerNorm Backward。
2. 同层对比：手写 Autograd Engine —— 用拓扑排序和局部 backward 函数实现一个最小自动微分框架。
3. 下游应用：Transformer 完整 Backward —— 逐项讲解 Attention、Softmax、RMSNorm、SwiGLU 的梯度与显存。

回复 1、2 或 3 继续；也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

本讲解使用稳定的微积分、矩阵微积分和 Reverse-Mode Automatic Differentiation 原理。PyTorch 示例遵循常见 eager-mode Autograd 语义；具体 `grad_fn`、Saved Tensor、Fused Backward、AOTAutograd、`torch.compile`、DDP Bucket 和 FSDP Hook 行为会随框架版本与编译路径变化。

生产环境中应以目标 PyTorch/CUDA/通信栈版本的源码、Profiler Timeline、数值对照和 Grad Check 为准。手算例子采用 FP32 小矩阵，旨在解释机制；它不代表大模型低精度训练中的舍入顺序、融合算子实现或分布式 Reduction 顺序。